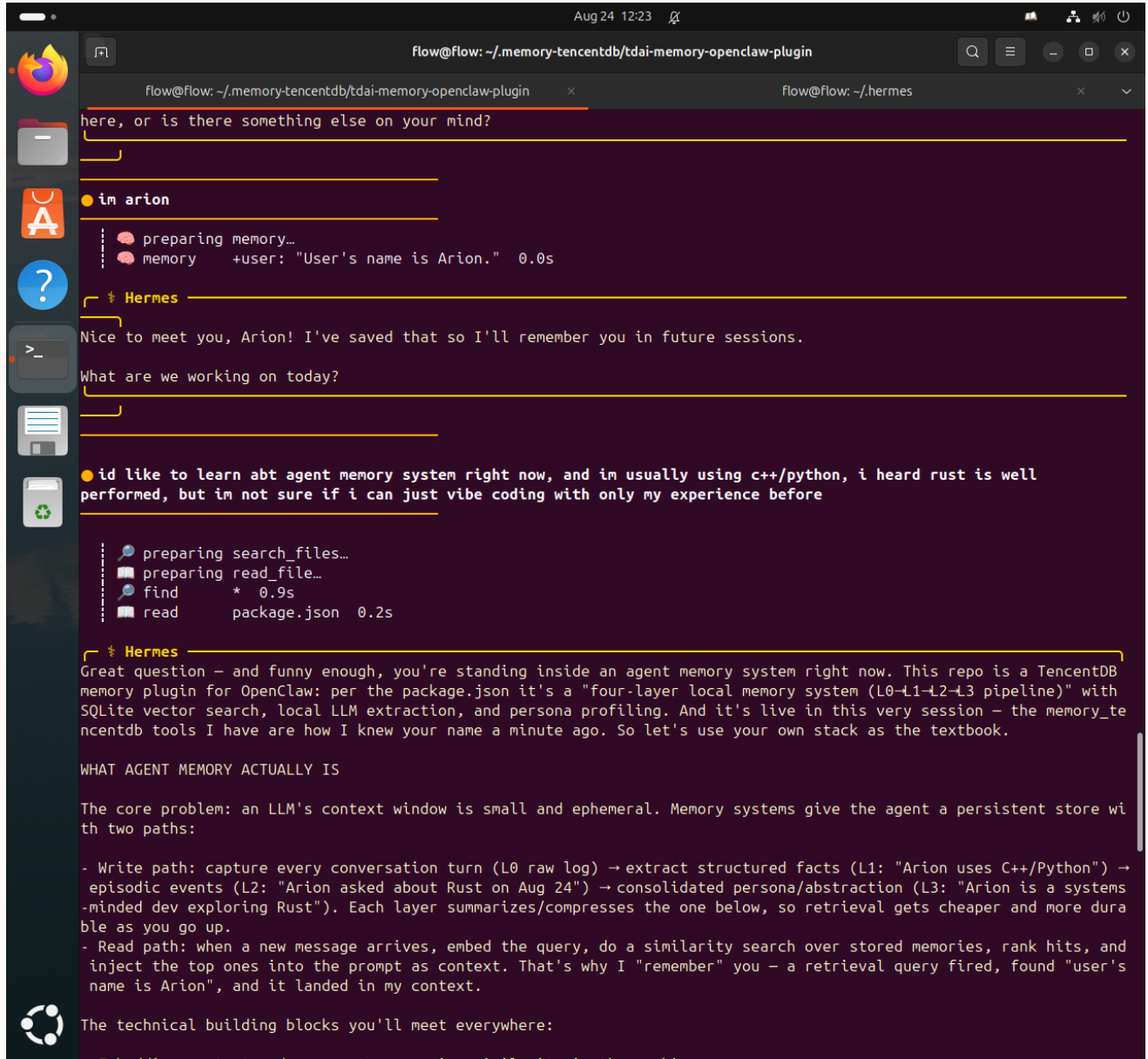


第一周验收标准（每人交）

- Hermes 成功对话 ≥ 1 轮（截图）。



```
flow@flow: ~/memory-tencentdb/tdai-memory-openclaw-plugin
here, or is there something else on your mind?

im arion

preparing memory...
memory +user: "User's name is Arion." 0.0s

Hermes
Nice to meet you, Arion! I've saved that so I'll remember you in future sessions.
What are we working on today?

id like to learn abt agent memory system right now, and im usually using c++/python, i heard rust is well
performed, but in not sure if i can just vibe coding with only my experience before

preparing search_files...
preparing read_file...
find * 0.9s
read package.json 0.2s

Hermes
Great question – and funny enough, you're standing inside an agent memory system right now. This repo is a TencentDB
memory plugin for OpenClaw: per the package.json it's a "four-layer local memory system (L0-L1-L2-L3 pipeline)" with
SQLite vector search, local LLM extraction, and persona profiling. And it's live in this very session – the memory_t
encentdb tools I have are how I knew your name a minute ago. So let's use your own stack as the textbook.

WHAT AGENT MEMORY ACTUALLY IS

The core problem: an LLM's context window is small and ephemeral. Memory systems give the agent a persistent store wi
th two paths:

- Write path: capture every conversation turn (L0 raw log) → extract structured facts (L1: "Arion uses C++/Python") →
episodic events (L2: "Arion asked about Rust on Aug 24") → consolidated persona/abstraction (L3: "Arion is a systems
-minded dev exploring Rust"). Each layer summarizes/compresses the one below, so retrieval gets cheaper and more dura
ble as you go up.
- Read path: when a new message arrives, embed the query, do a similarity search over stored memories, rank hits, and
inject the top ones into the prompt as context. That's why I "remember" you – a retrieval query fired, found "user's
name is Arion", and it landed in my context.

The technical building blocks you'll meet everywhere:
Embedding, text, docs, vectors, vector similarity, the vector...
```

- 能证明插件已被加载。

```
flow@flow:~/hermes$ curl -s http://127.0.0.1:8420/health | jq
{
  "status": "ok",
  "version": "0.1.0",
  "uptime": 7574,
  "stores": {
    "vectorStore": true,
    "embeddingService": false
  },
  "services": {
    "timerScanner": {
      "scansCompleted": 15127,
      "tasksEnqueued": 0,
      "scanErrors": 0,
      "lastScanMs": 0,
      "lastScanAt": 1787548264129,
      "isLeader": true,
      "nodeId": "scanner-mt6nknf0-ju16"
    },
    "pipelineWorker": {
      "tasksConsumed": 6,
      "tasksCompleted": 6,
      "tasksFailed": 0,
      "tasksRetried": 0,
      "tasksDeadLettered": 0,
      "lockConflicts": 0,
      "lockRenewFailed": 0,
      "lockLostDuringExecution": 0,
      "executionAborted": 0,
      "workerId": "worker-mt6nknh9-wf14",
      "deadLetterCount": 0
    },
    "stateBackend": "connected"
  }
}
```

- 能证明对话后记忆真的落库，有记忆插件生成的L0-L3层记忆（Gateway 日志有 capture / 数据目录下有内容）。

```
flow@flow:~/hermes$ ls ~/.memory-tencentdb/memory-tdai/
conversations  persona.md  records  scene_blocks  tdai-gateway.json  vectors.db  vectors.db-shm  vectors.db-wal
```

- L0

```
flow@flow:~/hermes$ cat ~/.memory-tencentdb/memory-tdai/conversations/2026-08-24.jsonl | jq .
{
  "sessionKey": "20260824_110441_3311f4",
  "sessionId": "",
  "recordedAt": "2026-08-24T03:04:49.783Z",
  "id": "msg_1787540689782_3389df",
  "role": "user",
  "content": "hello",
  "timestamp": 1787540689782
}
{
  "sessionKey": "20260824_110441_3311f4",
  "sessionId": "",
  "recordedAt": "2026-08-24T03:04:49.783Z",
  "id": "msg_1787540689782_502050",
  "role": "assistant",
  "content": "Hello! I'm Hermes, ready to help. I see we're in the tdai-memory-openclaw-plugin repo - want me to work on something here, or is there something else on your mind?",
  "timestamp": 1787540689782
}
{
  "sessionKey": "20260824_110441_3311f4",
  "sessionId": "",
  "recordedAt": "2026-08-24T03:04:59.069Z",
  "id": "msg_1787540699069_fcb375",
  "role": "user",
  "content": "im arion",
  "timestamp": 1787540699069
}
{
  "sessionKey": "20260824_110441_3311f4",
  "sessionId": "",
  "recordedAt": "2026-08-24T03:04:59.069Z",
  "id": "msg_1787540699069_60bb5f",
  "role": "assistant",
  "content": "Nice to meet you, Arion! I've saved that so I'll remember you in future sessions.\n\nWhat are we working on today?",
  "timestamp": 1787540699069
}
{
  "sessionKey": "20260824_110441_3311f4",
  "sessionId": "",
  "recordedAt": "2026-08-24T03:11:59.620Z",
  "id": "msg_178754119619_9cc229",
  "role": "assistant",
  "content": "id like to learn abt agent memory system right now, and im usually using c++/python, i heard rust is well performed, but im not sure if i can just vibe coding with only my experience before",
  "timestamp": 178754119618
}
{
  "sessionKey": "20260824_110441_3311f4",
  "sessionId": "",
  "recordedAt": "2026-08-24T03:11:59.620Z",
  "id": "msg_178754119619_9cc229",
  "role": "assistant",
  "content": "Great question - and funny enough, you're standing inside an agent memory system right now. This repo is a TencentDB memory plugin for OpenClaw: per the package.json it's a \"four-layer local memory system (L0-L1-L2-L3 pipeline)\" with SQLite vector search, local LLM extraction, and persona profiling. And it's live in this very session - the memory_tencentdb tools I have are how I knew your name a minute ago. So let's use your own stack as the textbook.\n\nWHAT AGENT MEMORY ACTUALLY IS\n\nThe core problem: an LLM's context window is small and ephemeral. Memory systems give the agent a persistent store with two paths:\n\n- Write path: capture every conversation turn (L0 raw log) -> extract structured facts (L1: \"Arion uses C++/Python\") -> episodic events (L2: \"Arion asked about Rust on Aug 24\") -> consolidated persona/abstraction (L3: \"Arion is a systems-minded dev exploring Rust\"). Each layer summarizes/compresses the one below, so retrieval gets cheaper and more durable as you go up.\n\n- Read path: when a new message arrives, embed the query, do a similarity search over stored memories, rank hits, and inject the top ones into the prompt as context. That's why I \"remember\" you - a retrieval query fired, found \"user's name is Arion\", and it landed in my context.\n\nThe technical building blocks you'll meet everywhere:\n\n- Embeddings - text -> dense vectors; cosine similarity is the ranking core\n\n- Vector search - exact scan or ANN indexes (HNSW, IVF); sqlite-vec here, or managed DBs (this plugin can offload to ClickHouse/Redis/Kafka, and TencentDB itself)\n\n- Hybrid retrieval - vector + keyword (BM25). Notice @node-rs/jieba in the deps: Chinese tokenization for keyword search\n\n- Memory typing - persona / episodic / instructional, each with different write policy and decay rules\n\n- Consolidation & forgetting - summarization, dedup, recency/importance scoring. Memory is as much about deleting as storing\n\n- Storage layering - hot (context) -> warm (local SQLite) -> cold (offload)\n\nRUST: CAN YOU VIBE IT?\n\nShort answer: yes, and agent memory is actually one of the best domains to do it in. The hot paths - tokenization, embedding inference, vector indexing - are exactly where Rust earns its keep. The ecosystem is real: Qdrant (vector DB) is Rust, Tantivy (search engine) is Rust, jieba-rs is a port of the very library in this repo's deps, and sqlite-vec itself sits in the C/Rust neighborhood.\n\nWith C++ behind you, the learning curve is much gentler than from Python-only:\n\n- Ownership/borrowing maps almost 1:1 to RAII, unique_ptr, and thinking about who owns memory - you already think in lifetimes, you just didn't name them\n\n- The borrow checker is basically the compiler being a strict reviewer of the habits you already have\n\nThe honest caveats for vibe coding:\n\n- The compiler is pedantic. Cargo build rejects code Python would happily run, and AI-generated Rust misses borrow/trait rules more often than generated Python. You'll spend real time on error messages.\n\n- The friction areas: async (tokio), trait objects + generics, self-referential structures, Send/Sync. Avoid those until you're comfortable.\n\n- Mitigations that work: 'cargo check' after every few lines (it's fast and the errors are famously good), rust-analyzer in your editor, 'cargo clippy' for style, and rustlings for the first week. Don't fight the borrow checker - restructure instead of cloning or adding unsafe.\n\nPractical path, if you want it: pick one narrow piece of THIS pipeline and reimplement it in Rust - e.g., a tiny vector index over embeddings, or the dedup/consolidation job. It's a small, well-scoped slice of a system you already have working as a reference, and you'll learn ownership, traits, and cargo in one shot without designing anything from scratch.\n\nWant me to walk through the actual L0-L3 pipeline code in this repo so you can see how the layers work in practice? That would ground the theory in the real implementation before you touch Rust.",
  "timestamp": 178754119619
}
```

○ L1

```
flow@flow:~/.hermes$ # 查看 L1 Records
cat ~/.memory-tencentdb/memory-tdai/records/* 2>/dev/null | jq .
{
  "id": "m_1787541149937_eebc3b7b",
  "content": "The user (Arion) goes by the name Arion.",
  "type": "persona",
  "priority": 90,
  "scene_name": "I (AI) am greeting the user and checking in on their needs",
  "source_message_ids": [
    "l0_20260824_110441_3311f4_1787540699071_0_9c0363"
  ],
  "metadata": {},
  "timestamps": [
    "2026-08-24T03:12:29.939Z"
  ],
  "createdAt": "2026-08-24T03:12:29.939Z",
  "updatedAt": "2026-08-24T03:12:29.939Z",
  "sessionKey": "20260824_110441_3311f4",
  "sessionId": ""
}
{
  "id": "m_1787541149937_0e785e77",
  "content": "The user (Arion) usually uses C++ and Python as his programming languages.",
  "type": "persona",
  "priority": 70,
  "scene_name": "I (AI) am introducing Arion to agent memory systems and discussing whether Rust suits him",
  "source_message_ids": [
    "l0_20260824_110441_3311f4_1787541119626_0_fd0c0c"
  ],
  "metadata": {},
  "timestamps": [
    "2026-08-24T03:12:29.942Z"
  ],
  "createdAt": "2026-08-24T03:12:29.942Z",
  "updatedAt": "2026-08-24T03:12:29.942Z",
  "sessionKey": "20260824_110441_3311f4",
  "sessionId": ""
}
{
  "id": "m_1787541149937_bf85efa2",
  "content": "The user (Arion) on 2026-08-24 expressed an interest in learning about agent memory systems and is considering learning Rust for performance, though he is unsure whether he can 'vibe code' in Rust with only his C++/Python"
}
```

○ L2

```
flow@flow:~/.hermes$ # 查看 L2 Scene Blocks
cat ~/.memory-tencentdb/memory-tdai/scene_blocks/* 2>/dev/null | jq .
jq: parse error: Invalid numeric literal at line 2, column 0
flow@flow:~/.hermes$ cat ~/.memory-tencentdb/memory-tdai/scene_blocks/* 2>/dev/null
-----META-START-----
created: 2026-08-24T03:12:39.954Z
updated: 2026-08-24T03:12:39.954Z
summary: Arion is a developer working mainly with C++ and Python. On 2026-08-24 he showed interest in agent memory systems and is considering learning Rust for performance, though unsure whether his existing experience transfers.
heat: 1
-----META-END-----

## User Basic Info
- Name: Arion
- Primary programming languages: C++ and Python

## User Core Traits
Arion is a C++/Python developer with a performance-oriented mindset. When a new domain attracts him (agent memory systems), he reacts by evaluating tooling and efficiency rather than staying passive, yet he is cautious about committing to a new language until he is confident it fits his existing workflow.

## User Preferences
- Primary languages: C++, Python
- Areas of interest: agent memory systems; Rust as a potential performance-focused addition
- Values: performance and a smooth, low-friction coding experience ("vibe code")

## Implicit Signals
- The phrase "vibe code" suggests Arion relies on a fluent, intuitive coding flow and worries that Rust's steeper learning curve could disrupt his productivity.
- His interest in agent memory systems may hint at involvement with AI agents or LLM-based applications, where memory architecture is a core concern.
- Choosing Rust for performance while coming from C++/Python shows a systems-level orientation and openness to lower-level tooling.

## Core Narrative
On 2026-08-24, Arion – a developer whose daily work centers on C++ and Python – came across the topic of agent memory systems and grew curious about it (Trigger). To pursue performance in that space, he began considering learning Rust (Action). However, he is uncertain whether he can "vibe code" in Rust with only his C++/Python background, so the decision remains open rather than committed (Result).

## Evolution Trajectory
- (No major shifts recorded yet.)

## To Confirm / Contradictions
- Whether Arion will actually start learning Rust, and in what concrete context he plans to apply agent memory systems, is not yet clear.
```

○ L3

```
Flow@Flow:~/.hermes$ # 查看 L3 Persona 画像
cat ~/.memory-tencentdb/memory-tdai/persona.md
# User Narrative Profile

> **Archetype**: A performance-oriented systems developer who evaluates every new tool through the twin lenses of efficiency and workflow fluency before committing.

> **Basic Info**
- Name: Arion
- Role: Software developer
- Primary languages: C++, Python

> **Long-term Preferences**
- Performance-driven tooling choices
- Low-friction, intuitive coding flow ("vibe code")
- Interest in agent memory systems / AI-adjacent applications

## 📖 Chapter 1: Context & Current State

Arion's daily work centers on C++ and Python. On 2026-08-24 he came across agent memory systems, grew curious, and reacted by evaluating tooling and efficiency rather than staying passive. To pursue performance in that space, he is now considering learning Rust – but the decision remains open, because he is not yet confident it fits his existing workflow.

## 🧑 Chapter 2: The Texture of Life

His coding style is fluency-driven: he values a smooth, intuitive flow ("vibe code") and low friction above all. His C++/Python foundation plus openness to Rust reveals a systems-level orientation and comfort with lower-level tooling. New domains attract him through their performance potential, yet adoption only proceeds when the path feels seamless.

## 🗨️ Chapter 3: Interaction & Cognitive Protocol

### 3.1 How to Speak
- Be concrete about performance trade-offs; frame advice around efficiency and measurable gains.
- Acknowledge his "vibe code" preference – address friction directly (e.g., Rust's learning curve, borrow checker) instead of dismissing it.

### 3.2 How to Think
- He weighs performance gains against workflow disruption; commitment comes only after confidence that a tool integrates cleanly.
- Validate whether his C++/Python experience transfers before encouraging adoption – he values honest feasibility assessments.

## 🌱 Chapter 4: Deep Insights & Evolution
```